

# Nettoyage et Exploration d'un Dataset Environnemental (Data Science)

## Objectifs de l'Atelier

- 1. Maîtriser les étapes fondamentales du *Data Pre-processing* (Nettoyage/Préparation) :** Importation, gestion des formats, traitement des valeurs manquantes et des aberrantes (*outliers*).
- 2. Appliquer l'Analyse Exploratoire de Données (EDA) en Python :** Utiliser les librairies pour obtenir des statistiques et des visualisations clés.
- 3. Générer un rapport d'exploration semi-automatisé** pour synthétiser l'état du jeu de données.

| Outil             | Utilisation                                                                                                                                                     |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Environnement     | Google Colaboratory (Colab)                                                                                                                                     |
| Librairies Python | <b>Pandas</b> (pour la manipulation des données), <b>Matplotlib/Seaborn</b> (pour la visualisation), <b>ydata-profiling</b> (pour l'automatisation du rapport). |

## Phase 1 : Importation et Inspection

| Étape                | Consignes Python Clés                               | Justification (Pourquoi le fait-on ?)                                                                                                    |
|----------------------|-----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Importation       | <code>import pandas as pd</code>                    | Le premier réflexe de tout Data Scientist.                                                                                               |
| 2. Chargement        | <code>df = pd.read_csv('nom_du_fichier.csv')</code> | Importation du fichier "fil rouge" de Google Drive ou GitHub.                                                                            |
| 3. Inspection        | <code>df.head(), df.info(), df.describe()</code>    | Obtenir une vue rapide des 5 premières lignes, vérifier les types (dtypes) et les statistiques descriptives (moyenne, écart-type, etc.). |
| 4. Nettoyage de Base | <code>df.columns = df.columns.str.lower()</code>    | Uniformisation des noms de colonnes (pas d'espace, en minuscules) pour la programmation.                                                 |

## Phase 2 : Gestion des Erreurs et des Aberrations

| Étape                                       | Consignes Python Clés                                                                                                          | Justification (Pourquoi le fait-on ?)                                                                                    |
|---------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <b>1. Valeurs Manquantes</b>                | <code>df.isnull().sum()</code>                                                                                                 | Identifier combien de valeurs (NaN) manquent par colonne.                                                                |
| <b>2. Traitement des Valeurs Manquantes</b> | <code>df['col'].fillna(df['col'].median(), inplace=True)</code> <b>ou</b> <code>df.dropna(subset=['col'], inplace=True)</code> | Choix stratégique : remplacer par la médiane (imputation) ou supprimer la ligne/colonne (si trop de valeurs manquantes). |
| <b>3. Gestion des Outliers</b>              | Utiliser des graphiques (boîtes à moustaches/scatter plot) ou la <b>méthode IQR</b> (Intervalle Interquartile).                | Les valeurs aberrantes peuvent fausser l'entraînement du modèle. Il faut les identifier et les neutraliser/supprimer.    |
| <b>4. Vérification des Formats</b>          | Forcer les colonnes de dates ( <code>pd.to_datetime</code> ), et les colonnes numériques ( <code>astype(float)</code> ).       | Essentiel pour que le modèle puisse interpréter correctement les données.                                                |

## Phase 3 : Visualisations Exploratoires Avancées

Utiliser **Seaborn** pour des visualisations rapides qui mettent en lumière les relations entre les variables.

- Distribution** : Créer un **histogramme** (`sns.histplot`) pour visualiser la répartition de la variable cible (ex: niveau de risque, consommation énergétique).
- Corrélation** : Calculer et visualiser la **matrice de corrélation** entre toutes les variables numériques (`sns.heatmap`). Identifier les variables fortement liées.
- Relation** : Utiliser un **nuage de points** (`sns.scatterplot`) pour visualiser la relation entre deux variables importantes (ex: Température vs Émissions de polluants).

#### **Phase 4 : Rapport d'Exploration Automatisé (30 min)**

Pour gagner du temps et synthétiser l'analyse, utiliser la librairie `ydata-profiling` :

1. **Installation :** `pip install ydata-profiling`
2. **Génération du Rapport :** `profile = df.profile_report(title="Rapport d'Exploration IA")`
3. **Affichage :** `profile.to_notebook_iframe()` (pour afficher le rapport interactif directement dans Colab).

# Les Trois Stratégies pour le Traitement des NaN

3 méthodes pour gérer les valeurs manquantes, chacune ayant ses avantages et ses risques :

## 1. La Suppression (Deletion) ❌

- **Principe** : Retirer la ligne entière (l'observation) si elle contient une ou plusieurs valeurs NaN.
- **Avantages** : Méthode la plus simple et la plus rapide. Le jeu de données restant est parfaitement propre.
- **Inconvénients et Risques** : Si beaucoup de lignes sont supprimées, cela peut entraîner une **perte significative d'information** et biaiser le modèle (le modèle n'apprendra pas sur les cas qui avaient des données manquantes).

## 2. L'Imputation (Imputation) 💡

- **Principe** : Remplacer la valeur NaN par une valeur calculée. C'est la méthode la plus courante.
- **Méthodes courantes d'Imputation** :
  - **Moyenne (Mean)** : Remplacer le NaN par la moyenne de tous les autres nombres dans cette colonne.
  - **Médiane (Median)** : Remplacer le NaN par la valeur médiane (préférable si la colonne contient des *outliers*).
  - **Mode (Mode)** : Remplacer le NaN par la valeur la plus fréquente (utilisé pour les données catégorielles).
- **Avantages** : Permet de conserver toutes les observations (lignes) du jeu de données, maximisant ainsi l'information disponible pour l'entraînement.

## 3. La Modélisation (Model-Based Imputation) 🤖

- **Principe** : Utiliser un autre algorithme de Machine Learning pour **prédire** la valeur manquante, en se basant sur les autres variables de la ligne.
- **Avantages** : Méthode potentiellement la plus précise car elle utilise le contexte des autres données pour estimer la valeur manquante.
- **Inconvénients et Risques** : C'est la méthode la plus complexe à mettre en œuvre et elle nécessite d'entraîner un modèle secondaire.

⚠️ **Choix de l'Ingénieur** : La décision de supprimer, d'imputer par la médiane, ou d'utiliser une méthode plus complexe est un choix critique qui dépend de la **proportion de NaN** et du **type de variable** concernée (une NaN dans la température n'a pas la même implication qu'une NaN dans le PIB).